Low-Level Design (LLD)

RAG-Based Customer Support Assistant

Using LangGraph & HITL

Submitted by: Roshini Parvathi

Batch: Feb 2026

Organization: Innomatics Research Labs

Date: 23/04/2026

TABLE OF CONTENTS

📄 ✅ LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant (LangGraph + Groq)

---

1. Introduction

This document describes the internal implementation details of the RAG-based customer support assistant. It covers modules, data structures, workflows, and system logic.

---

2. Module-Level Design

1. Document Processing Module

- Loads PDF using `PyPDFLoader`
- Extracts raw text from pages

---

2. Chunking Module

- Splits document into structured sections using regex
- Each section corresponds to a logical topic (Refund, Account, etc.)

---

3. Embedding Module

- Uses `HuggingFaceEmbeddings`
- Converts text chunks into vector representations

---

4. Vector Storage Module

- Uses `ChromaDB`
- Stores embeddings with metadata:
    - source
    - chunk_id

---

5. Retrieval Module

- Uses similarity search
- Retrieves top-k relevant chunks based on query

---

6. Query Processing Module

- Accepts user input
- Formats prompt using retrieved context

---

7. Graph Execution Module (LangGraph)

- Controls execution flow
- Nodes:
    - `process`
    - `route`
    - `hitl`

---

8. HITL Module

- Triggered when:
    - No context found
    - LLM says “I don’t know”
- Returns escalation response

---

3. Data Structures

Document Structure

```
{
 "page_content": "text",
 "metadata": {
   "source": "knowledge_base.pdf",
   "chunk_id": 1
 }
}
```

Graph State Object

```
{
 "query": "string",
 "context": ["chunk1", "chunk2"],
 "answer": "string",
 "confidence": 0.7,
 "route": "ANSWER/HITL",
 "sources": []
}
```

Source Structure

```
{
 "source": "knowledge_base.pdf",
 "chunk_id": 2,
 "preview": "Refund policy...",
 "score": 2
}
```

---

4. Workflow Design (LangGraph)

Nodes

- **Process Node**
  - Retrieves documents
  - Calls LLM
  - Generates answer
- **Route Node**
  - Decides next step
  - Based on answer/context
- **HITL Node**
  - Returns escalation response

Edges

- process → route
- route → answer / hitl
- hitl → end

State Flow

- Query enters process node
- Context + answer generated
- Route decides final output

---

5. Conditional Routing Logic

Answer Condition

- Context available
- LLM generates valid response

Escalation Condition

- No context retrieved
- OR answer contains “I don’t know”

---

6. HITL Design

Trigger Conditions

- Low relevance
- Missing information
- User asks for human

Flow

1. Query reaches route node
2. Routed to HITL
3. Returns:

   ⚠️ Escalated to human agent

Integration

- HITL response replaces LLM answer
- No sources returned

7. API / Interface Design

Input

```
{
 "query": "user question"
}
```

Output

```
{
 "answer": "response",
 "confidence": 0.7,
 "sources": []
}
```

Interaction Flow

1. User enters query
2. System processes via LangGraph
3. Answer displayed with citation

---

8. Error Handling

Cases Covered

- ❌ No PDF uploaded
  → Show warning
- ❌ No relevant chunks
  → Trigger HITL
- ❌ LLM failure
  → Return error message
- ❌ Empty query
  → Ignore request

---

9. Performance Considerations

- Efficient retrieval using top-k
- Reduced latency using Groq API
- Chunk size optimized for context quality